

Spec Driven Development (SDD)

SDD adaptado a Flutter + Supabase · OpenSpec · Integración FADER

Qué es SDD

METODOLOGÍA

Spec Driven Development es escribir especificaciones primero, generar código después.

La idea central: (*rigidez a cambio*) se detecta en specs antes de que se convierta en bugs.

OpenSpec (openspec.dev · 65.7k ☆ · MIT) es el tool que almacena specs y las consulta de forma optimizada para agentes de IA.

Instalación

```
npm install -g @fission-ai/openspec@latest
```

Las 4 fases de SDD

WORKFLOW

Fase	Objetivo	Output
Diseño	Definir qué construir	Specs aprobadas
Desarrollo	Generar código que cumpla specs	Feature completa
Validación	Verificar que specs se cumplen	Tests + Validación
Mantenimiento	Actualizar specs al cambiar	Specs actualizadas

Los 3 Gates

Gate	Cuándo	Criterio
1: Diseño	Antes de código	Specs aprobadas
2: Implementación	Antes de merge	Código cumple specs + tests pasan
3: Validación	Antes de release	<code>openspec validate</code> pasa

OpenSpec en la práctica

TOOLING

Comandos esenciales

```
openspec init      # Crear .openspec/
openspec propose   # Generar specs candidatas
openspec exec "task" # Ejecutar tarea con contexto optimizado
openspec status    # Ver uso de contexto
openspec scan      # Chequeo de salud
```

Workflow diario

```
1. openspec doctor      # salud del proyecto
2. openspec exec "tarea" # ejecutar con contexto
3. openspec status      # verificar contexto
4. openspec scan        # integridad cada 2-3 commits
```

Specs por capa

Capa	Tipo de spec	Ejemplo
Domain	Entidad / UseCase / Repo (interfaz)	User tiene id UUID, email, displayName
Infrastructure	Datasource / Repo impl	SupabaseDatasource usa signInWithPassword()
Presentation	Cubit / State	AuthCubiti: initial → loading → authenticated
UI	Página / Widget	LoginEmailPage usa listener_builder

Especificaciones EARS (patrones)

Patrón	Ejemplo Flutter
WHEN [evento]	WHEN presiona Submit, llamar loginUseCase
IF [trigger]	IF email inválido, mostrar error en campo
WHERE [componente]	WHERE AuthCubiti, emitir AuthError en excepción
WHILE [condición]	WHILE no autenticado, redirigir a /login

Boundary rules

Boundary	Regla
Domain → Infrastructure	Domain define interfaces; Infrastructure las implementa
Infrastructure → Supabase	Solo datasources tocan Supabase client
Presentation → Domain	Cubits usan UseCases, nunca Datasources
UI → Presentation	Páginas usan BlocBuilder, nunca lógica de negocio

Integración SDD + FADER

Flujo integrado (7 pasos)

1 Alcance openspec propose	2 FADER Refinar specs	3 Contratos Especs EARS	4 Flujo Spec UI + Cubit
5 Backend Spec datasource	6 Criterios openspec validate	7 Scaffold openspec exec	

Cuándo usar cada tool

Situación	Tool
Feature nueva desde cero	FADER + OpenSpec + clean-arch-feature
Feature existente, agregar componente	Solo clean-arch-component + SDD lite
Fix de bug	Solo spec del componente afectado
Feature core con integración externa	FADER completa + SDD + OpenSpec validate

Profundidad de spec según feature

Tipo	Profundidad	Ejemplo
CRUD simple	Básica	Formulario de contacto
Feature core	Completa (EARS + boundaries)	Login con Supabase Auth
Integración externa	Alta (mocks + contratos)	Pago con Stripe
Fix de bug	Mínima	Corregir cálculo

Anti-patrones

Anti-patrón	Consecuencia	Solución
Specs vagas	Código ambiguo	Usar EARS específico
Saltar Gate 1	Features erróneas	Aprobar specs antes de código
Specs monolíticas	Difícil mantener	1 spec por componente
No actualizar specs	Specs desactualizadas	Actualizar en cada PR relevante

Recuerda: OpenSpec es un *tool*. SDD es una *methodology*. Usar OpenSpec sin SDD = contexto optimizado sin estructura. Usar SDD sin OpenSpec = methodology sin herramientas.

Módulo 23 · Spec Driven Development (SDD)

Complementa *Guía-SDD-equipos-ágiles.pdf* · OpenSpec + Flutter + Supabase + FADER